

Real Estate CRM SaaS — Architecture Document

Product: Multi-tenant CRM for Indian real estate organizations (brokerages and/or developers) — projects, contacts, leads, activity history, one-tap call logging. **Model:** SaaS, sold to multiple independent real estate organizations. Each organization is a tenant; tenants never see each other's data.

1. Market Reality (India-specific)

Salespeople are mobile-first and move between site visits and calls all day — any workflow that requires sitting at a laptop to log activity will be ignored. Mobile web or a lightweight app has to be the primary surface, not an afterthought.

WhatsApp is the default communication channel with leads. The CRM doesn't need to replace WhatsApp — it needs to make logging what happened on WhatsApp/calls fast enough that salespeople actually do it.

Google Sheets is the current default "CRM" for most small brokerages. Anything you build is competing with a tool that's free, familiar, and infinitely flexible — the pitch has to be "less typing, not more structure."

Connectivity is inconsistent on site visits (under-construction sites, basements, remote plots). The mobile client should tolerate being offline for logging at minimum (queue and sync), even if full offline CRUD is out of scope for V1.

Cost sensitivity is high for small brokerages (2-20 salespeople); per-seat pricing needs to be low (₹200-500/user/month range is typical for this segment) or you price per-organization with seat tiers.

Existing India-specific competitors (LeadSquared, Sell.Do, generic Zoho setups) are established — differentiate on low-friction logging and the project/contact-role model, not on having "AI."

2. Product Model — Entities & Relationships



Organization (tenant)

└── Region (optional, org-scoped)

└── User (salesperson / manager / admin — belongs to org, optionally a region)

└── Project (a development/property — belongs to org)

| └── ProjectContact (join: Project ↔ Person, with role:

owner/builder/architect/engineer/guard/other)

└── Person (base identity record — phone-normalized, dedup anchor; can be a lead, buyer, or project stakeholder)

└── Lead (a sales opportunity — Person × Project, tracked through Pipeline)

└── PipelineStage (org-configurable, ordered)

└── Activity (unified timeline — call/meeting/site_visit/note; linked to Lead and/or Project)

└── Task/Follow-up (linked to Lead or Activity, has due date + owner)

└── Document (photos, brochures — linked to Project or Lead)

└── AuditLog (who changed what, when)

Why Person is separate from Lead: the same phone number shows up as a project's site guard, a past lead, and sometimes a repeat buyer. Deduplicating on `Person` (keyed by normalized phone number, per org) keeps one identity across all those roles instead of scattering the same contact across disconnected records — this directly matches your Part 17 concern about duplicate contacts.

Why a Project isn't owned by one salesperson: multiple salespeople interact with the same project over time (confirmed in your source doc — this is correct and I'm keeping it). Ownership of a *Lead* (a specific buyer's journey) can be assigned to one salesperson at a time; ownership of a *Project* itself should be team-visible, not person-locked.

Relationship history: every Activity is a timestamped row, not a field that gets overwritten. The "last conversation" field problem you flagged in your source doc is solved by never having that field — the timeline view is a query (`activities where lead_id = X order by occurred_at desc`), not a stored summary.

3. Call Capture — Final Design (verified against current Play Store policy)

Automatic call-log capture requires the app to be registered as the user's **default**

Phone/SMS/Assistant handler under current Google Play policy, and even then usage must be "absolutely essential" to the app's primary function — a CRM doesn't qualify, and Google actively removes apps that request `READ_CALL_LOG` without qualifying. iOS has no API for reading call history at all.

V1 design (what's actually shippable):

Android: listen for call-state changes ([TelephonyCallback], uses the *unrestricted* [READ_PHONE_STATE] permission, not the restricted call-log one). When a call ends, pop an in-app prompt: "Log call with [matched contact / this number]?" with 3-4 outcome buttons (interested / call back / not interested / site visit booked). One tap, not zero.

iOS: no auto-trigger possible. A persistent "Log a call" quick-action on the home screen, pre-filled with the most recently dialed/received number from the OS call history the user can see (not one you can read programmatically) — user picks it manually. This is a real limitation to be upfront about, not engineered around.

Both platforms: manual "+ Log activity" is always available regardless of the above.

You've already said this is a nice-to-have, not the core differentiator — good, because it caps at "fast manual logging," not "silent auto-capture," on both platforms.

4. Lead Sources / Integrations

Source	Official API	Notes
Manual entry	—	Always available, zero-risk baseline for V1
WhatsApp	WhatsApp Business Platform (Cloud API)	Needs Meta Business verification; per-conversation pricing; webhook-based inbound
Website	Your own form → webhook	Trivial, build first
Meta Ads (Instagram/Facebook lead forms)	Meta Lead Ads API	Needs Meta app review; webhook-based; leads expire from Meta's side if not polled/subscribed promptly
Instagram DMs	Instagram Messaging API (via Meta)	More restrictive review process than lead ads; treat as V2, not MVP

None of these are MVP-blocking except manual entry + website form. WhatsApp and Meta Ads integrations should be V1/V2 — they involve real approval lead time (days to weeks) that shouldn't block your first paying customer.

5. AI/ML — What's Justified, What Isn't

Feature	Verdict	Why
Lead message parsing have real lead volume to justify the (Hinglish → location/budget/type) dropdowns solves 90% of this at zero marginal cost in V1.	Defer to V2. Use it only once you	Value is real but not urgent — a salesperson can fill 4 fields faster than you can build and validate an extraction pipeline.
Natural-language CRM search ("projects not contacted in 30 days")	Skip the LLM. Use filters/saved views.	This is a SQL query with a UI, not an AI problem. Don't add an LLM-to-SQL layer for something a dropdown filter does more reliably and with zero prompt-injection surface.
Follow-up suggestions	Rules-based for V1 (e.g. "no activity in 7 days on an open lead → suggest follow-up"), not ML.	You don't have historical data yet to train or even meaningfully prompt against.
Lead scoring	Skip entirely until you have hundreds of closed-won/lost leads per org.	No AI/ML model — including LLM-based scoring — beats "stage + days since last contact" with zero historical data.
Meeting/call summarization	Legitimate V2 AI use case once call transcripts exist (e.g. from a WhatsApp voice note or a recorded meeting note) — real value, moderate complexity.	This is the one feature where AI beats structured alternatives, because the input is genuinely unstructured text.

Bottom line for AI in V1: none. Every AI feature you listed is more valuable once you have real usage data, and none of them are needed to prove the core product works. Building AI features before proving the CRM core is used is the most common way this kind of project stalls.

6. Sales Pipeline & Lost-Lead Reactivation

Stages should be **org-configurable**, not hardcoded — a developer selling one large project needs different stages (Enquiry → Site Visit → Booking → Agreement → Registration) than a brokerage juggling many projects (New → Contacted → Qualified → Site Visit → Negotiation → Won/Lost). Ship with a sensible default stage set per org on signup, editable from settings.

Lost-lead reactivation: configurable interval per organization (default suggestion: 60 days), stored as an org setting, not hardcoded. A scheduled job flags leads that hit `lost_at + reactivation days` and creates a follow-up task — doesn't need AI, just a daily cron check.

7. Google Sheets Decision

Option B: PostgreSQL as source of truth, Sheets as export/sync — not primary storage.

Reasoning: this is multi-tenant with concurrent writes from multiple salespeople per org. Sheets has no real concurrency control, no referential integrity, and no practical row-level security between tenants — using it as primary storage for a product you're *selling* to multiple orgs is a data-loss and cross-tenant-leak risk, not just a technical inconvenience. Offer CSV/Sheets *export* for managers who want to keep their own copy — that's a feature, not an architecture.

8. n8n / Automation — Where It Belongs

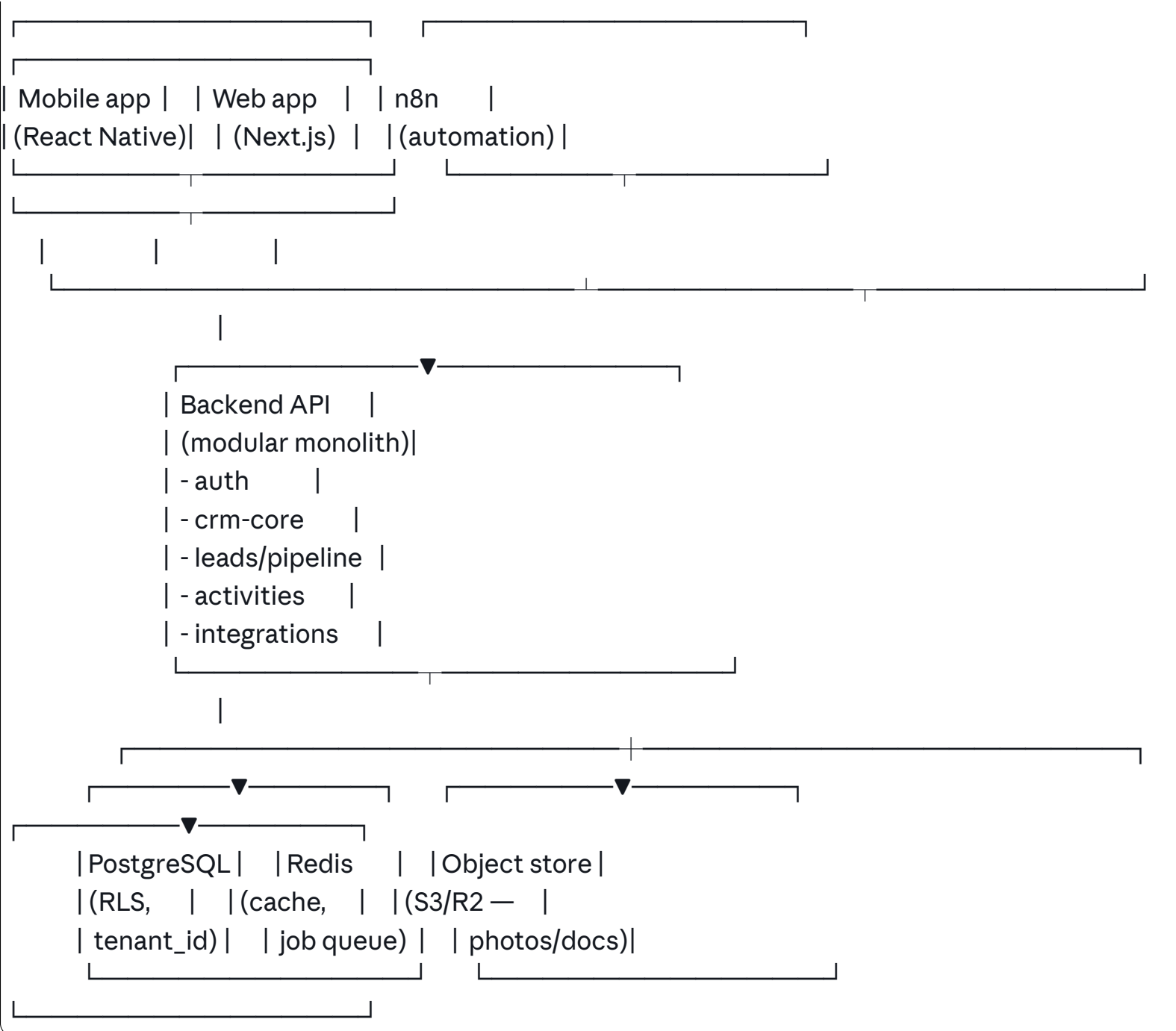
n8n owns: WhatsApp/Meta webhook ingestion → normalize → push into your backend via API (not directly into the DB), scheduled reactivation-check triggers, notification delivery (WhatsApp/email/push), Sheets export jobs.

Backend owns: everything that decides business state — pipeline stage transitions, lead assignment logic, tenant isolation checks, validation. Keep n8n workflows thin (ingest → call backend API → done) so your core rules live in one testable place, not scattered across visual workflows.

9. Architecture

Modular monolith. No microservices, no Kubernetes, no Kafka — nothing here justifies that complexity at 10, 100, or even 1,000 tenant orgs. A single well-structured backend service with clear internal module boundaries (auth, crm-core, leads, activities, integrations) scales to tens of thousands of users on a single properly-sized Postgres instance with read replicas added later if needed.





10. Technology Selection

Layer	Choice	Why	Rejected alternative
Backend	FastAPI (Python) or Node/NestJS	Either is fine; pick whichever you're stronger in — this isn't a place to learn a new backend language on a commercial project	Go/Rust — no need for that performance ceiling at this scale, slower to build in
Database	PostgreSQL, single schema, <code>tenant_id</code> on every table + Row-Level	RLS gives you a hard backstop against IDOR/tenant-leak bugs in application code — cheap insurance for a multi-tenant	Schema-per-tenant — unmanageable migrations once you have 50+ tenants; MongoDB — you have genuinely relational data

	Security	SaaS	(projects↔contacts↔leads↔activities)
Cache/queue	Redis	Job queue for background tasks (reactivation checks, notification sends) + simple caching	Kafka/RabbitMQ — no throughput here justifies them
Frontend (web)	Next.js	Standard, good ecosystem, fine for a manager-facing dashboard	—
Mobile	React Native	One codebase for Android+iOS call-state listener + quick-log UI; native module needed for the Android <code>TelephonyCallback</code> piece	Flutter — comparable choice, pick based on your familiarity
Auth	Custom JWT or Clerk/Auth.js, with org_id embedded in the token	Needs to carry tenant context into every request cleanly	—
Storage	Cloudflare R2 or AWS S3	Photos, brochures, documents	—
Automation	n8n (self-hosted)	Matches your stated preference, appropriate use per section 8	—
LLM (V2 only)	Whichever provider — deferred, see section 5	—	—

11. Data Model — Core Tables

organizations: id, name, plan, reactivation_days (config), created_at

users: id, org_id (FK), region_id (FK, nullable), name, phone, role (admin/manager/salesperson), created_at

regions: id, org_id (FK), name

people: id, org_id (FK), phone_normalized (unique per org), name, source, created_at — dedup anchor

projects: id, org_id (FK), name, location, developer_name, status, created_at

project_contacts: id, project_id (FK), person_id (FK), role (owner/builder/architect/engineer/guard/other), notes

pipeline_stages: id, org_id (FK), name, sort_order

leads: id, org_id (FK), person_id (FK), project_id (FK, nullable), stage_id (FK), assigned_to (user_id, FK), budget, property_type, timeline, status (open/won/lost), lost_at, created_at

activities: id, org_id (FK), lead_id (FK, nullable), project_id (FK, nullable), person_id (FK), user_id (FK), type (call/meeting/site_visit/note), occurred_at, duration_seconds, outcome, notes

tasks: id, org_id (FK), lead_id (FK), assigned_to (user_id, FK), due_at, status, created_from_activity_id (nullable)

documents: id, org_id (FK), project_id (FK, nullable), lead_id (FK, nullable), file_url, type

audit_log: id, org_id (FK), actor_id (FK), action, entity_type, entity_id, diff (jsonb), created_at

Indexes: every table indexed on `org_id` (leading column of composite indexes — RLS predicate hits this constantly); `people.phone_normalized` unique index scoped to `org_id`; `leads (org_id, assigned_to, stage_id)` for pipeline board queries; `activities (org_id, lead_id, occurred_at desc)` for timeline queries.

Phone normalization: store E.164 format (`+91xxxxxxxxxx`), strip spaces/dashes/leading zeros at write time — this is what prevents the duplicate-contact problem your source doc flagged.

12. Security — MVP / Pre-Production / Can Wait

Required for MVP:

Tenant isolation via `org_id` on every query + Postgres RLS as a backstop (don't rely on application code alone)

Password hashing (bcrypt/argon2), JWT with short expiry + refresh tokens

Basic authorization checks (role-based: admin/manager/salesperson) on every endpoint

Input validation on all write endpoints (SQL injection is a non-issue with a parameterized ORM, but validate anyway)

HTTPS everywhere, no exceptions

Required before charging real customers:

Rate limiting on auth endpoints and public webhook endpoints

Webhook signature verification (WhatsApp/Meta send signed payloads — verify them, don't trust the body blindly)

File upload validation (type/size limits, no executable uploads, virus scan if budget allows)

Audit log on sensitive actions (lead reassignment, data export, user role changes)

Automated backups with tested restore procedure — not just "backups exist," but "we've restored one and it worked"

Secrets in environment variables / a secrets manager, never in code or n8n workflow JSON

Can wait:

Full penetration testing (do a lightweight self-review first; pay for a real pentest once you have paying customers with real data)

SOC2/formal compliance (not needed to sell to Indian SMBs early on)

Advanced anomaly detection / SIEM

AI security (once V2 AI features land): never give the LLM direct database write access — structured function calls with server-side validation only; strip PII before sending lead text to a third-party LLM provider where feasible; treat any lead-submitted text as untrusted input (prompt injection surface) if it ever reaches an LLM prompt.

13. MVP / V1 / Production Scope

MVP (prove the core loop works, get first design-partner org onboarded): Org + user auth, projects, project contacts (with roles), people/lead dedup, manual lead creation, configurable pipeline stages, activity timeline (manual logging only, no call-state listener yet), tasks/follow-ups, basic web dashboard + mobile-responsive web (skip native app for MVP — ship native later).

V1 (what a real paying customer needs): Native mobile app with one-tap call logging (Android call-state listener + iOS manual quick-log), website lead form → webhook, CSV/Sheets export, configurable lost-lead reactivation, basic reporting (leads by stage, by salesperson).

Production (reliable commercial operation): WhatsApp Business API integration, Meta Lead Ads integration, audit log UI for managers, rate limiting + webhook verification hardened, tested backup/restore, multi-region support if a customer needs it, basic usage-based billing.

14. Cost Estimate (early stage — 10-50 tenant orgs)

Hosting (backend + Postgres, e.g. Railway/Render/Hetzner): ~\$20-60/month at this scale

Object storage (R2/S3): ~\$5-15/month depending on photo volume

WhatsApp Business API: per-conversation pricing, roughly ₹0.35-₹0.90 per conversation depending on category — this scales with usage, budget for it once V1 ships

Meta Ads API: free to call, cost is only your ad spend as a customer (not yours)

n8n: self-hosted, just the VM cost (often shares the backend VM early on)

LLM (deferred to V2): budget separately once you actually add it — don't provision for a feature you're not building yet

Most expensive line item at real scale will be WhatsApp conversation fees, not compute — this is a variable cost that should probably be passed through or factored into your per-seat pricing.

15. Biggest Risks

Call capture disappointment — if it was pitched to prospects as "automatic," the one-tap reality needs to be set correctly in sales conversations from day one, not discovered after signup.

WhatsApp/Meta approval delays — these aren't instant; don't let V1 launch depend on them being approved on schedule.

Tenant data leakage — the single highest-severity bug class for this product; RLS + tests for it are not optional polish.

Building AI before proving the CRM core gets used — the most common way projects like this stall.

Underestimating manual-entry fatigue — if logging a call/site visit takes more than ~10 seconds, salespeople stop doing it and the product becomes another abandoned tool, same as the generic CRMs it's replacing.

Phone number normalization edge cases — Indian numbers with/without +91, landlines, shared family numbers — get this wrong and lead dedup silently breaks.

16. What I Would Build vs Not Build

Build: the CRM core (projects/contacts/leads/activities/pipeline) properly, one-tap call logging, manual entry + website form as the only V1 lead sources, configurable pipeline and reactivation windows, RLS-backed multi-tenancy from day one.

Not build (yet): any LLM feature, native call-log auto-capture, Instagram DM integration, microservices/k8s, schema-per-tenant, lead scoring.

Why: every "not build" item either has no data to justify it yet (AI features), isn't reliably deliverable (auto call capture), or adds operational complexity with no scale justification (microservices, schema-per-tenant). The product's actual differentiator — fast logging + real project/contact modeling — doesn't need any of them.

Assumption made in this document

Solo or small-team build (based on your background), so the stack choices favor one backend/one frontend developer being able to own the whole system rather than assuming a dedicated DevOps or mobile specialist. Flag it if that's wrong — it changes the MVP-vs-V1 split.